

databases-normalize

Normalizacion de bases de datos

1. [Dependencias funcionales \(una introduccion a la normalizacion\)](#)
 - [Tipos de dependencias funcionales](#)
 - [Las reglas de inferencia de Armstrong y sus aplicaciones](#)
 2. [Normalizacion](#)
 - [Formas normales](#)
 - [Primera forma normal](#)
 - [Segunda forma normal](#)
 - [Tercera forma normal](#)
 - [BCNF Forma Normal de Boyce Codd](#)
 - [Cuarta forma normal](#)
 - [Quinta forma normal](#)
 - [RESUMEN Y PRACTICAS](#)
-

Dependencias_funcionales

Cuatro criterios informales de calidad:

1. Semántica clara de atributos
2. Reducir redundancia de información
3. Minimizar valores NULL
4. Evitar tuplas falsas (spurious tuples)

las tuplas falsas son registros que aparecen normalmente al hacer JOIN y son datos que no existen

Supongamos que divides tu información en estas dos tablas:

Tabla Persona:

CURP	nombreP	NomProyecto
88888	Ana	Proyecto 1
55555	Juan	Proyecto 2

Tabla Proyecto:

#P	nombreProy
21	Proyecto 1
55	Proyecto 2
32	Proyecto 2

← ¡Hay DOS proyectos con el mismo nombre!

Cuando haces un **JOIN por nombre de proyecto**:

```
SELECT * FROM Persona NATURAL JOIN Proyecto;
```

Obtienes:

CURP	nombreP	nombreProy	#P	
88888	Ana	Proyecto 1	21	✓ Correcto
55555	Juan	Proyecto 2	55	✓ Correcto
55555	Juan	Proyecto 2	32	✗ ¡TUPLA FALSA!

¿Qué es una Dependencia Funcional?

Una **dependencia funcional** $X \rightarrow Y$ significa:

"Si dos tuplas coinciden en los valores de X, entonces DEBEN coincidir en los valores de Y"

Es como decir: "**X determina a Y**" o "**Y depende de X**"

R representa el **esquema de una relación** (tabla) en el modelo relacional.

Cuando decimos:

" $X \rightarrow Y$ en R"

Significa:

- En la relación R
- El conjunto de atributos X determina funcionalmente a Y

ejemplo:

$R = \text{Préstamo}(\text{numPréstamo}, \text{nombreSucursal}, \text{nombreCliente}, \text{importe})$

$\text{numPréstamo} \rightarrow \text{importe} \quad (\text{en } R)$

Esto se lee: "En la relación Préstamo, numPréstamo determina funcionalmente a importe"

r (minúscula) = INSTANCIA o EXTENSIÓN

- Los datos actuales
- Las tuplas/filas concretas

Ejemplo:

$R = \text{Empleado}(\text{CURP}, \text{Nombre}, \text{Sueldo})$

DF: $\text{CURP} \rightarrow \text{Nombre}$

Esto significa que en CUALQUIER estado válido de Empleado:

Estado r_1 (válido):

CURP	Nombre	Sueldo
ABC123	Ana	10000
DEF456	Juan	15000
ABC123	Ana	12000

✓ Mismo CURP $\rightarrow$ mismo Nombre

Estado r_2 (inválido):

CURP	Nombre	Sueldo
ABC123	Ana	10000
ABC123	Pedro	15000

x Mismo CURP $\rightarrow$ diferente Nombre

Tipos_de_dependencias_funcionales

Dependencia Funcional Completa: Y depende funcionalmente de X, pero no de ningún subconjunto propio de X.

Dependencia Funcional Parcial: Y depende de un subconjunto propio de X (donde X es una clave compuesta).

Dependencia Funcional Transitiva: Si $X \rightarrow Y$ y $Y \rightarrow Z$, entonces $X \rightarrow Z$ (pero Y no debe ser clave candidata).

1. DF Triviales

Son obvias y siempre se cumplen:

- $\{A, B, C\} \rightarrow A$ (el lado derecho está contenido en el izquierdo)
- $CURP \rightarrow CURP$

2. DF No Triviales

Son las interesantes para el diseño:

- $CURP \rightarrow \{\text{Nombre, Dirección, FechaNac}\}$
- $\text{NumPréstamo} \rightarrow \text{Importe}$

3. DF que definen Llaves

Una llave es un conjunto de atributos que:

- Determina TODOS los demás atributos, i.e. que su cerradura son todos los atributos
- Es **mínimo** (no puedes quitar ninguno)

una **superllave**: es un conjunto de atributos que contienen una llave, i.e. puede tener atributos extra innecesarios

sobre la minimalidad

- Si tienes una llave candidata $\{A, B, C\}$
- Entonces NINGUNO de estos subconjuntos puede ser llave por sí solo:
 - $\{A\}$
 - $\{B\}$
 - $\{C\}$
 - $\{A, B\}$
 - $\{A, C\}$
 - $\{B, C\}$

Ejemplo con

Relación: Estudiante

```
Estudiante(NumCuenta, CURP, Nombre, Carrera, Promedio)
```

Opción 1: ¿{NumCuenta, CURP} es llave?

Paso 1: Verificar que determina todo

```
{NumCuenta, CURP}+ = {NumCuenta, CURP, Nombre, Carrera, Promedio}
```

Sí determina todos los atributos

Paso 2: Verificar MINIMALIDAD (ningún subconjunto propio es llave)

Probar subconjunto {NumCuenta}:

```
{NumCuenta}+ = {NumCuenta, Nombre, Carrera, Promedio, CURP}  
= TODOS los atributos
```

✗ {NumCuenta} solo YA es llave

Conclusión: **{NumCuenta, CURP} NO es llave candidata** porque tiene un atributo innecesario (CURP). Es una **superllave**.

Reglas_de_inferencia_de_Armstrong

IMPORTANTE

Estas reglas te permiten **derivar nuevas DFs a partir** de las conocidas:

R1. Reflexividad

```
Si  $Y \subseteq X$ , entonces  $X \rightarrow Y$   
Ejemplo:  $\{A, B, C\} \rightarrow A$ 
```

ejemplitos

```
AB  $\rightarrow$  A   ✓  
AB  $\rightarrow$  B   ✓
```

ABC $\rightarrow$ A ✓
ABC $\rightarrow$ B ✓
ABC $\rightarrow$ C ✓
ABC $\rightarrow$ AB ✓
ABC $\rightarrow$ AC ✓
ABC $\rightarrow$ BC ✓
ABCD $\rightarrow$ A ✓
ABCD $\rightarrow$ ABC ✓

R2. Aumento

Si $X \rightarrow Y$, entonces $XZ \rightarrow YZ$
Ejemplo: Si $A \rightarrow B$, entonces $AC \rightarrow BC$

R3. Transitividad

Si $X \rightarrow Y$ y $Y \rightarrow Z$, entonces $X \rightarrow Z$
Ejemplo: Si $A \rightarrow B$ y $B \rightarrow C$, entonces $A \rightarrow C$

Reglas Derivadas (para simplificar)

R4. Descomposición

Si $X \rightarrow YZ$, entonces $X \rightarrow Y$ y $X \rightarrow Z$
Ejemplo: Si $A \rightarrow BC$, entonces $A \rightarrow B$ y $A \rightarrow C$

R5. Unión

Si $X \rightarrow Y$ y $X \rightarrow Z$, entonces $X \rightarrow YZ$
Ejemplo: Si $A \rightarrow B$ y $A \rightarrow C$, entonces $A \rightarrow BC$

Cerradura de conjuntos de dependencias funcionales

Son otro conjunto de dependencias (regals) que se implican o inferen por F (el conjutno original)

Cerradura de Atributos (X^+)

Concepto clave: X^+ es el conjunto de TODOS los atributos que puedes determinar a partir de X.

Si Z pertenece a la cerradura

NO requiere que cada elemento individual determine Z.

Algoritmo Paso a Paso

Entrada: Conjunto X, Conjunto de DFs F

Salida: X^+

1. $X^+ := X$ (empiezas con X mismo)
2. Repetir hasta que no cambien:
Para cada DF " $Y \rightarrow Z$ " en F:
Si Y está completamente contenido en X^+ :
Agregar Z a X^+
3. Regresar X^+

es decir que vamos checando cada pedacito de X

Ejemplo Completo

Dado:

- $R(A, B, C, D, E)$
- $F = \{A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow A\}$

Calcular: $\{A\}^+$

Iteración 0:

$X^+ = \{A\}$

Iteración 1:

$A \rightarrow BC$ ($A \subseteq \{A\}$ ✓) $\rightarrow X^+ = \{A, B, C\}$

Iteración 2:

$B \rightarrow D$ ($B \subseteq \{A, B, C\}$ ✓) $\rightarrow X^+ = \{A, B, C, D\}$

Iteración 3:

$CD \rightarrow E$ ($C, D \subseteq \{A, B, C, D\}$ ✓) $\rightarrow X^+ = \{A, B, C, D, E\}$

No hay más cambios.

Resultado: $\{A\}^+ = \{A, B, C, D, E\}$

Conclusión: Como A determina todos los atributos, **A es llave candidata**.

Normalizacion

Se puede motivar partiendo la relacion en dos tablas por separadas cuya union (con un join) da de nuevo toda la tabla

Para la normalizacion se desea:

- Eliminar las anomalias.
- Que las relaciones fraccionadas tengan un join sin perdida.
(Recuperacion de la informacion)
- Conservar las dependencias funcionales.

Formas_normales

Entre mas alta la forma normal mas seguros son los datos

1NF

Primera forma normal (1NF)

Esta se define en cuestion de las tablas de realcion

Una relación está en Primera Forma Normal si y solo si todos sus atributos contienen valores atómicos, es decir, no existen grupos repetitivos ni atributos multivaluados.

Requisitos

1. Cada columna debe contener valores atómicos (indivisibles)
2. Los valores en una columna deben ser del mismo dominio
3. Cada columna debe tener un nombre único
4. El orden de las filas y columnas no debe ser relevante
5. No deben existir filas duplicadas (debe existir una clave primaria)

Ejemplos

Formas de violar esta forma normal:

- Apoyarse en el orden de las filas para inferir informacion (como las alturas a partir de nombres de personas por el orden de las filas)
- Combinar tipos de datos

- Crear tablas sin llave primaria
- Tener grupos repetidos en algun atributo o en la misma fila (como inventario en un string, hay cascos, espadas, etc.)
 - Tener diferentes calificaciones o grupos en una misma tabla (habra muchos NULL)

Soluciones

- Crear tablas especificas para una informacion en particular (crear una tabla con un llave y otro atributo alturas en tipo numeric, etc.)
- Crear aparte una tabla de tipos de grupos y luego relacionarlos con otras (como jugador con su inventario que se relaciona con los diferentes grupos muchos a muchos)

Tabla No Normalizada:

Empleado_ID	Nombre	Teléfonos
101	Juan	555-1234, 555-5678
102	María	555-9876

Tabla en 1NF:

Empleado_ID	Nombre	Teléfono
101	Juan	555-1234
101	Juan	555-5678
102	María	555-9876

2NF

Segundo forma normal (2NF)

Son aquellas que nos evitan anomalias

Una relación está en Segunda Forma Normal si está en 1NF y todos los atributos no-clave dependen funcionalmente de forma completa de la clave primaria (no existen dependencias parciales). Es decir que los atributos no llave no dependen de UNA PARTE DE LA LLAVE (mas formalmente de LLAVE CANDIDATA)

Requisitos

1. Debe estar en 1NF

2. No debe haber dependencias funcionales parciales de atributos no-clave respecto a la clave primaria
3. Solo aplica cuando la clave primaria es compuesta

Terminología

1. **Anomalía de Eliminación/Borrado (Deletion Anomaly):** Cuando al eliminar un registro, se pierde información no relacionada que debería mantenerse.
2. **Anomalía de Actualización (Update Anomaly):** Cuando hay redundancia y actualizar un dato requiere modificar múltiples registros (como se muestra con el Player_Rating de jdog21 que aparece dos veces).
3. **Anomalía de Inserción (Insertion Anomaly):** Cuando no se puede agregar información sin tener otros datos relacionados.

Esta forma normal se refiere a como los atributos (no llaves) se relacionan con la llave primaria **INFORMALMENTE**: nos dice que cada atributo no-llave debe depender completamente en la llave primaria

ejemplo

- Si no hacemos una tabla para alguna clasificación y los valores repetidos no hacen referencia una llave candidata de otra tabla, es decir que cada valor repetido es independiente de otro (sin CASCADE de otra tabla etc.)
- Si eliminamos las tuplas del inventario de un jugador (por ejemplo, todos los ítems de jdog21), perdemos también la información del Player_Rating, que es un atributo independiente del inventario y debería conservarse.

 Player_ID	Item_Type	Item_Quantity	Player_Rating
jdog21	amulets	2	Advanced
jdog21	rings	4	Intermediate
** deletion anomaly **			
trev73	shields	3	Advanced
trev73	arrows	5	Advanced
trev73	copper coins	30	Advanced
trev73	rings	7	Advanced
** update anomaly **			

Si desarrollamos las dependencias funcionales:

- Nos daremos cuenta que no podemos poner $\{Player_ID, Item_Type\} \rightarrow \{Player_Rating\}$ pues hay casos donde

$\{ \text{Player_ID}, \text{Item_Type} \} \rightarrow \{ \text{Item_Quantity} \}$ ✓
 $\{ \text{Player_ID} \} \rightarrow \{ \text{Player_Rating} \}$ ✗

solucion

Player

 Player_ID	Player_Rating
jdog21	Intermediate
gilal9	Beginner
trev73	Advanced
tina42	Beginner

$\{ \text{Player_ID} \} \rightarrow \{ \text{Player_Rating} \}$

Tabla en 1NF pero no en 2NF:

SUPONIENDO QUE NO SEPARASTE ESTA RELACION EN OTRAS TABLAS INDEPENDIENTES

IE QUE SOLO TIENES LA INFO DE ESTUDIANTE Y CURSO EN ESTA RELACION

Estudiante_ID	Curso_ID	Nombre_Estudiante	Nombre_Curso	Calificación
1001	CS101	Ana López	Programación	95

1001	CS102	Ana López	Algoritmos	88

Dependencias funcionales:

- (Estudiante_ID, Curso_ID) → Calificación ✓ (dependencia completa)
- Estudiante_ID → Nombre_Estudiante ✗ (dependencia parcial)
- Curso_ID → Nombre_Curso ✗ (dependencia parcial)

Solución en 2NF:

Tabla Estudiantes:

Estudiante_ID	Nombre_Estudiante
1001	Ana López

Tabla Cursos:

Curso_ID	Nombre_Curso
CS101	Programación
CS102	Algoritmos

Tabla Inscripciones:

Estudiante_ID	Curso_ID	Calificación
1001	CS101	95
1001	CS102	88

Problemas que Resuelve

- Elimina anomalías de actualización relacionadas con dependencias parciales
- Reduce redundancia de información
- Separa entidades independientes

Otros conceptos fundamentales al analizar

Storage_Locker_Reservations

Locker_ID	Reservation_Start_Date	Reservation_End_Date	Reservation_End_Day
221	14-MAY-2019	12-JUN-2019	Wednesday
308	07-JUN-2019	12-JUN-2019	Wednesday
537	14-MAY-2019	17-MAY-2019	Friday

Candidate key = an attribute (or combination of attributes) that uniquely identifies a row in the table.

Prime attribute = an attribute that belongs to at least one candidate key.

Non-prime attribute = an attribute that doesn't belong to any candidate key.

Our candidate keys:

- 1) {Locker_ID, Reservation_Start_Date}
- 2) {Locker_ID, Reservation_End_Date}

Our prime attributes:

- 1) Locker_ID, 2) Reservation_Start_Date, 3) Reservation_End_Date

Our non-prime attribute(s):

Reservation_End_Day

podríamos decir que esa tabla no está en segunda forma normal pues `start_date` → `end_date` i.e. si tenemos la misma fecha de inicio NO ES CIERTO esa dependencia no es cierta

3NF

Tercera forma normal (3NF)

Una relación está en Tercera Forma Normal si está en 2NF y no existen dependencias transitivas de atributos no-clave respecto a la clave primaria.

INTUITIVAMENTE: cualquier atributo no llave debería depender en solo la llave

1. Todos los atributos no-clave deben depender directa y únicamente de la clave primaria

Tabla en 2NF pero no en 3NF:

Empleado_ID	Nombre	Departamento_ID	Nombre_Departamento	Ubicación_Depto
-----	-----	-----	-----	-----
E001	Carlos	D10	Ventas	Piso 3
E002	Laura	D20	IT	Piso 5
E003	Roberto	D10	Ventas	Piso 3

Dependencias funcionales:

- Empleado_ID → Departamento_ID (directa)
- Departamento_ID → Nombre_Departamento (transitiva)

- Departamento_ID → Ubicación_Depto (transitiva)

Solución en 3NF:

Tabla Empleados:

Empleado_ID	Nombre	Departamento_ID
E001	Carlos	D10
E002	Laura	D20
E003	Roberto	D10

Tabla Departamentos:

Departamento_ID	Nombre_Departamento	Ubicación_Depto
D10	Ventas	Piso 3
D20	IT	Piso 5

Ejemplo

No puede haber dependencias funcionales entre atributos no llaves

Player

 Player_ID	Player_Rating	Player_Skill_Level
jdog21	Intermediate	4
gila19	Beginner	4
trev73	Advanced	8
tina42	Beginner	1

Inconsistency!

SKILL LEVEL

1 2 3	4 5 6	7 8 9
Rating: Beginner	Rating: Intermediate	Rating: Advanced

$\{ \text{Player_ID} \} \rightarrow \{ \text{Player_Skill_Level} \}$

$\{ \text{Player_ID} \} \rightarrow \{ \text{Player_Skill_Level} \} \rightarrow \{ \text{Player Rating} \}$

solucion

Player		Player_Skill_Levels	
Player_ID	Player_Skill_Level	Player_Skill_Level	Player_Rating
jdog21	4	1	Beginner
gila19	4	2	Beginner
trev73	8	3	Beginner
tina42	1	4	Intermediate
		5	Intermediate
		6	Intermediate
		7	Advanced
		8	Advanced
		9	Advanced

BCNF

Una relación está en Forma Normal de Boyce-Codd si está en 3NF y para cada dependencia funcional $X \rightarrow Y$, X es una superclave.

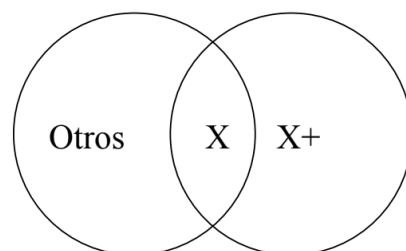
Para estar en BCNF, **cada** dependencia funcional no trivial debe tener una superllave en el lado izquierdo.

es muy parecida a la tercera forma normal solo que:

***INTUITIVAMENTE:** cualquier atributo debería depender en solo la llave (NO IMPORTA QUE SEA NO LLAVE)

La estrategia a seguir es:

- 1 Buscar una DF no trivial $X \rightarrow B$ que viole BCNF.
- 2 Calcular X^+ .
- 3 Fraccionar R en $R1(X^+)$ y $R2((R-X^+) \cup X)$.



- 4 Encontrar las DFs para las nuevas relaciones.

ejemplo:

La relación $P(\text{nombreSuc}, \text{nombreCliente}, \text{numPréstamo}, \text{importe})$
con $F = \{\text{numPréstamo} \rightarrow \text{importe nombreSuc}\}$

- ① Buscar una DF no trivial que viole BCNF.
- ② $\text{numPréstamo}^+ = \{\text{numPréstamo}, \text{importe}, \text{nombreSuc}\}$
- ③ Se descompone en:
 - $P1(\text{numPréstamo}, \text{importe}, \text{nombreSuc})$
 - $P2(\text{nombreCliente}, \text{numPréstamo})$
- ④ Encontrar las DF que se cumplen en cada relación.
 - $P1(\text{numPréstamo}, \text{importe}, \text{nombreSuc})$
 $F1 = \{\text{numPréstamo} \rightarrow \text{importe nombreSuc},$
 $\text{numPréstamo importe} \rightarrow \text{nombreSuc},$
 $\text{numPréstamo nombreSuc} \rightarrow \text{importe}\}$
 - $P2(\text{nombreCliente}, \text{numPréstamo})$
- ⑤ Revisar si las nuevas relaciones están en BCNF.
- ⑥ Resultado: $P = P1 \bowtie P2$, con $F1$ en $P1$

Formas de encontrar dependencias funcionales en subrelaciones

Para calcular las DF mantenidas en $R1$ hacer:

- ① Para cada conjunto X de atributos contenidos en $R1$ calcular X^+ .
- ② Para cada atributo B de $R1$ que no esté en X y sí esté en X^+ se tiene la dependencia funcional $X \rightarrow B$ en $R1$.

Sean $R(A, B, C, D, E)$ y $F \{A \rightarrow D, B \rightarrow E, DE \rightarrow C\}$

Determinar las DF's que se cumplen en $S(A, B, C)$

- ① $\{A\}^+ = \{A, D\}$, como D no está en el esquema de S , no hay DF aquí.
- ② $\{B\}^+ = \{B, E\}$, no hay DF
- ③ $\{C\}^+ = \{C\}$, no hay DF
- ④ $\{A, B\}^+ = \{A, B, C, D, E\}$, con lo cual se obtiene la dependencia funcional $AB \rightarrow C$ para S .
- ⑤ Ningún otro par de atributos agrega nada nuevo.
- ⑥ El conjunto de $\{A, B, C\}$ no da dependencias no triviales para S .

Por tanto, la única dependencia para S es $AB \rightarrow C$.

Ejercicio

Normaliza

$$R(A,B,C,D) \text{ y } DF = \{A \rightarrow B, A \rightarrow C\}$$

De primera instancia la DF no viola ninguna

1. Calculamos

=

2. Tenemos ahora

y **MUY IMPORTANTE: cualquier relacion de dos atributos esta en BCNF**

3. Encontrar las DF de cada relacion

De son }

Las ultimas dos se obtuvieron pues por trivialidad

nota: por ejemplo si tenemos dependencias funcionales donde no haya llaves entonces diremos que viola la BCNF pues de por si ya viola la primera forma normal

Otro ejemplo para normalizar

Reservaciones (película, cine, ciudad) con
 $DF = \{cine \rightarrow ciudad, película \text{ ciudad} \rightarrow cine\}$

llaves candidatas

Como `cine` no es llave y esta del lado izquierdo de una dependencia funcional entonces no esta en BCNF

Creacion de la descomposicion

Solucion

- $R_1(cine, ciudad)$ con $cine \rightarrow ciudad$, y
- $R_2(cine, película)$

... Ejercicios de normalización BCNF

- ❶ $R(A,B,C,D)$ y $DF = \{A \rightarrow B, A \rightarrow C\}$
- ❷ $P(A,B,C,D,E,F,G)$ y $DF = \{AB \rightarrow CDE\}$
- ❸ $R(A,B,C,D)$ con $DF = \{AB \rightarrow C, BC \rightarrow D, CD \rightarrow A, AD \rightarrow B\}$
- ❹ $R(A,B,C,D,E)$ con $DF = \{AB \rightarrow C, C \rightarrow D, D \rightarrow E\}$
- ❺ $R(nSuc, nCliente, noPrestamo, importe)$ y:
 $DF: \{ noPrestamo \rightarrow importe \ nSuc \}$
- ❻ $R(nSuc, delegacion, activo, nCliente, noPrestamo, importe)$ y:
 $DF: \{ nSuc \rightarrow delegacion \ activo, noPrestamo \rightarrow importe \ nSuc \}$
 $R(S, D, A, C, P, I) F = \{S \rightarrow DA, P \rightarrow IS\}$

Atributos superfluos

Cuando verificamos si un atributo es superfluo, lo hacemos **respecto a UNA dependencia funcional específica**, no respecto a todo el conjunto F .

Atributo superfluo en el lado izquierdo (regla 1):

- Verificamos si un atributo A es superfluo en α (lado izquierdo) de la $DF \alpha \rightarrow \beta$
- Pregunta: ¿Podemos quitar A de α y aún así derivarla la misma dependencia usando las mismas dependencias?

Atributo superfluo en el lado derecho (regla 2):

- Verificamos si un atributo A es superfluo en β (lado derecho) de la $DF \alpha \rightarrow \beta$
- Pregunta: ¿Podemos quitar A de β y aún así derivar $\alpha \rightarrow A$ usando las demás dependencias reemplazando la original con con el que no tienen a A ?

A es un **atributo superfluo** si se puede eliminar de la DF sin que se altere la cerradura de F.

Sean $\alpha \rightarrow \beta$ una DF en F y A un atributo, A es superfluo si

- ① Si A está en α . Sea $\gamma = \alpha - \{A\}$ si $F \models \gamma \rightarrow \beta$.
- ② Si A está en β . Sea $F' = (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$, si $F' \models \alpha \rightarrow A$.

Ejemplos:

- ① Determinar los atributos superfluos de $F = \{AB \rightarrow C, A \rightarrow C\}$
 - ¿A es superfluo en $AB \rightarrow C$?
Se debe probar que $F \models B \rightarrow C$.
 $B^+ = \{B\}$ por tanto no es superfluo.
 - ¿B es superfluo en $AB \rightarrow C$?
Probar que $F \models A \rightarrow C$.
 $A^+ = \{AC\}$ Sí lo es.

Conjuntos de dependencias funcionales minimos

Un conjunto F de DFs es **mínimo** si

- No tiene atributos superfluos.
 - Cada lado izquierdo de las DF de F es único, es decir no existen $\alpha_1 \rightarrow \beta_1, \alpha_2 \rightarrow \beta_2$ tales que $\alpha_1 = \alpha_2$.
- ① Aplicar la regla de la unión a DFs tales que $\alpha_1 \rightarrow \beta_1, \alpha_1 \rightarrow \beta_2$ para obtener $\alpha_1 \rightarrow \beta_1\beta_2$ y sustituir con esta última las DF's con igual lado izquierdo.
 - ② Eliminar los atributos superfluos de las DFs.

Ejercicios

Sea $R(A,B,C)$ y $F = \{A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C\}$ determinar si F es mínimo y si no es así, encontrar el mínimo.

Algoritmo para obtener el minimo

- Unir cuando haya atributo con lado izquierdo igual

- verificar siempre si hay superfluos donde haya mas de dos atributos en algun lado de una dependencia
- Eliminar atributos superfluos en aquellos DFs donde haya varios atributos involucrados del lado izquierdo o derecho, i.e poner la misma dependencia pero si en el superfluo
- Si ya tienes el minimo solo tienes que crear una tabla por cada dependencia funcional
Indirectamente estas quitando las dependencias transitivas
- Recordar que debemos conservar la llave original de las dependencias originales F, entonces si faltan atributos en todas las tablas o esquemas para que se haga la superllave crear una sola tabla que contenga todos los atributos de la superllave

FINALMENTE

- 1 Hacer F mínimo (F_m).
- 2 Para toda DF en F_m crear una relación que contenga sólo los atributos de ella.
- 3 Si no existe esquema que contenga una superllave de R , crear otra relación cuyo esquema sea una llave de R .

con los atributos se refiere a los del lado derecho i izquierdo

EJEMPLO

¿ $R(A,B,C,D)$ con $F = \{A \rightarrow B, B \rightarrow C, AC \rightarrow D\}$ está en 3NF?

A es superfluo de $AC \rightarrow D$?

i.e. se puede derivar $C \rightarrow D$? no,

C es superfluo de $AC \rightarrow D$?

i.e. se puede derivar $A \rightarrow D$ con las mismas dependencias, partimos de A y vemos que obtenemos C entonces de ahí obtenemos D, por lo tanto si. Reducimos a

Tenemos una forma

Por ultimo queda verificar que B y D no son superfluos y así continuar hasta que verifiquemos todos

¿La relación $R(A,B,C,D,E)$ y $F = \{AB \rightarrow C, C \rightarrow D, D \rightarrow B, D \rightarrow E\}$ está en 3NF?

Llaves candidatas

$\{A,B\}, \{C,A\}, \{A,D\}$

Primero queremos encontrar el minimo (como indica el algoritmo)

Es A o B superfluo en $AB \rightarrow C$?

1. se puede derivar $B \rightarrow C$ con las originales (recordar que no cambiamos las dependencias originales): NO
 2. se puede derivar $A \rightarrow C$ con las originales? NO
Es B o E superfluo en $D \rightarrow BE$?
 3. se puede derivar $D \rightarrow B$ de $F - \{D \rightarrow BE\} \{D \rightarrow E\}$? NO
 4. se puede derivar $D \rightarrow E$ de $F - \{D \rightarrow BE\} \{D \rightarrow B\}$? NO
- Por lo tanto ya es el minimo

Serian tres relaciones? (A, B, C) (C, D) (D, B, E) correcto

¿ $R(A,B,C,D,E)$ y $F = \{AB \rightarrow C, C \rightarrow B, A \rightarrow D\}$ está en 3NF?

A o B son superfluos en $AB \rightarrow C$

1. Se puede derivar $B \rightarrow C$ con las originales? NO
2. Se puede derivar $A \rightarrow C$ con las originales? NO

② Dividir.

Se obtienen tres esquemas: $R_1(A,B,C)$, $R_2(C,B)$ y $R_3(A,D)$.

Se elimina R_2 por estar contenida en R_1 .

③ ¿Agregar una relación? Sí se agrega $R_4(A,B,E)$

El resultado es:

$R_1(A,B,C)$ con $AB \rightarrow C$ y $C \rightarrow B$,

$R_3(A,D)$ con $A \rightarrow D$ y

$R_4(A,B,E)$

tomamos el ultimo esquema pues las llaves candidatas era: $\{ABE\}$ y $\{ACE\}$

4NF

Cuarta forma normal (4NF)

Una relación está en Cuarta Forma Normal si está en BCNF y no contiene dependencias multivaluadas no triviales.

Dependencia Multivaluada (MVD)

Una dependencia multivaluada $X \twoheadrightarrow Y$ existe cuando para cada valor de X existe un conjunto de valores de Y independiente de otros atributos.

EJEMPLO

Tabla en BCNF pero no en 4NF:

Empleado	Habilidad	Idioma
Juan	Java	Inglés
Juan	Java	Francés
Juan	Python	Inglés
Juan	Python	Francés

Dependencias multivaluadas:

- Empleado $\twoheadrightarrow$ Habilidad (independiente de Idioma)
- Empleado $\twoheadrightarrow$ Idioma (independiente de Habilidad)

Solución en 4NF:

Tabla Empleado_Habilidad:

Empleado	Habilidad
Juan	Java
Juan	Python

Tabla Empleado_Idioma:

Empleado	Idioma
Juan	Inglés
Juan	Francés

Las anomalías que pudieron haber pasado

Anomalía de Inserción (Insertion Anomaly) - LA MÁS GRAVE

Escenario 1: Juan aprende un nuevo idioma: **Italiano**

Problema: Tienes que insertar MÚLTIPLES filas (una por cada habilidad que tenga

Anomalía de Eliminación

Escenario: Juan ya no quiere trabajar con Java

Problema: Tienes que eliminar MÚLTIPLES filas (una por cada idioma que tenga con Java)

Pero si solo sabia Java entonces tambien ya le quitaste todos sus idiomas

5NF

Quinta forma normal (5NF)

Una relación está en Quinta Forma Normal si está en 4NF y toda dependencia de unión (join dependency) es implicada por las claves candidatas.

Requisitos

1. Debe estar en 4NF
2. No debe existir descomposición sin pérdida que no sea implicada por las claves candidatas
3. La relación debe representar restricciones independientes que no pueden separarse sin pérdida de información semántica

Es como si lo descompusieras en varias proyecciones

Fifth normal form (5NF)

Available_Flavors_By_Brand

Brand	Flavor
Frosty's	Vanilla
Frosty's	Strawberry
Frosty's	Mint Chocolate Chip
Alpine	Vanilla
Alpine	Rum Raisin
Ice Queen	Vanilla
Ice Queen	Strawberry
Ice Queen	Mint Chocolate Chip

Preferred_Brands_By_Person

Person	Brand
Jason	Frosty's
Jason	Alpine
Suzy	Alpine
Suzy	Ice Queen

Preferred_Flavors_By_Person

Person	Flavor
Jason	Vanilla
Jason	Chocolate
Suzy	Rum Raisin
Suzy	Mint Chocolate Chip
Suzy	Strawberry

Fifth Normal Form:
The table (which must be in 4NF) cannot be describable as the logical result of joining some other tables together.

```
select
  pbrand.Person,
  bf.Brand,
  bf.Flavor
from
  Preferred_Brands_By_Person pbrand
inner join
  Preferred_Flavors_By_Person pflavor
on
  pbrand.Person = pflavor.Person
inner join
  Available_Flavors_By_Brand bf
on
  pbrand.Brand = bf.Brand
  and pflavor.Flavor = bf.Flavor
```

Recursos Académicos en Línea

1. **Stanford University - Database Course Materials:** cs.stanford.edu/people/widom/DB-mooc.html
2. **MIT OpenCourseWare - Database Systems:** ocw.mit.edu
3. **Database Systems - The Complete Book** (Garcia-Molina, Ullman, Widom)
4. **W3Schools SQL Tutorial - Normalization:** w3schools.com/sql
5. **Oracle Database Documentation - Data Modeling:** docs.oracle.com

Resumen_formas_normales

Forma Normal	Elimina	Basada en
1NF	Grupos repetitivos, atributos multivaluados	Atomicidad
2NF	Dependencias parciales	Dependencias funcionales completas
3NF	Dependencias transitivas	Dependencias directas a la clave
BCNF	Dependencias funcionales donde el determinante no es superclave	Superclaves como determinantes
4NF	Dependencias multivaluadas	Independencia de conjuntos de valores
5NF	Dependencias de unión complejas	Descomposición sin pérdida óptima

Estándares y Documentación Técnica

- 1. **ISO/IEC 9075 (SQL Standard):** Estándar internacional para bases de datos SQL
- 2. **ANSI Database Standards:** Documentación de estándares americanos
- 3. **IEEE Standards Association - Database Standards**